

Unidad 7

Modelado de Aspectos dinámicos

Hasta aquí se llegaron a **representar estáticamente el sistema**, las distintas **entidades** del dominio, sus **atributos y las asociaciones** entre estas por medio de diagramas de clase y objeto de UML. Luego, **OCL permitió agregar las distintas restricciones que el dominio impone** para que tanto la estructura del futuro del sistema, como las operaciones que debe soportar sean consistentes.

Si retrocedemos un poco más atrás, encontramos los Casos de Uso (CU) que resultaron del análisis de los requerimientos elicitados, para el modelado de estos hemos utilizado diagramas de CUs y fichas para especificarlos. Estos **CUs modelan las interacciones entre los distintos actores externos y el sistema**. Ahora que el sistema va adquiriendo una arquitectura y comienza a notarse la estructura interna es necesario no solo modelar las interacciones Actor-Sistema, sino ir un poco más allá y comenzar a representar como serán las interacciones de todas las entidades involucradas en la ejecución de las distintas operaciones que dan soporte al comportamiento requerido por los CUs.

Diagramas de interacción

Para continuar el análisis, luego del modelado estático es necesario comenzar a pensar como los objetos, instancias de las clases que se pudieron identificar, interactúan para realizar el comportamiento especificado en un CU.

Para modelar los aspectos dinámicos necesitamos de los modelos Estáticos y de CUs, con estos como guía utilizamos diagramas de interacción para modelar explícitamente como las instancias de las clases colaboran e interactúan para realizar alguno o todos los comportamientos de un CU. Existen 4 tipos de diagramas de interacción: diagramas de secuencia, diagramas de comunicación, diagramas globales de interacción y diagramas de tiempo, en el curso utilizaremos sólo los diagramas de secuencia.

Diagramas de secuencia del sistema

Para comprender mejor la utilidad y elaboración de los diagramas de secuencia, en principio se prescindirá de los modelos estáticos (diagramas de clases) comenzando con la utilización de Diagramas de Secuencia del Sistema (DSS). Este tipo de diagrama de actividad modela interacciones de actores externos con el sistema, visto este como una “caja negra”. A partir de este planteo se puede deducir que los DSS nos permiten modelar distintos escenarios para un CU determinado, de esta forma podemos tener una traza directa entre un CU y un conjunto de DSS que representen los distintos escenarios para ese CU (Figura 1).

Un DSS es un diagrama que muestra, para un escenario específico de un caso de uso (CU), los eventos que generan los actores externos y las respuestas a estos que devuelve el sistema, todo esto ordenado en una secuencia temporal.

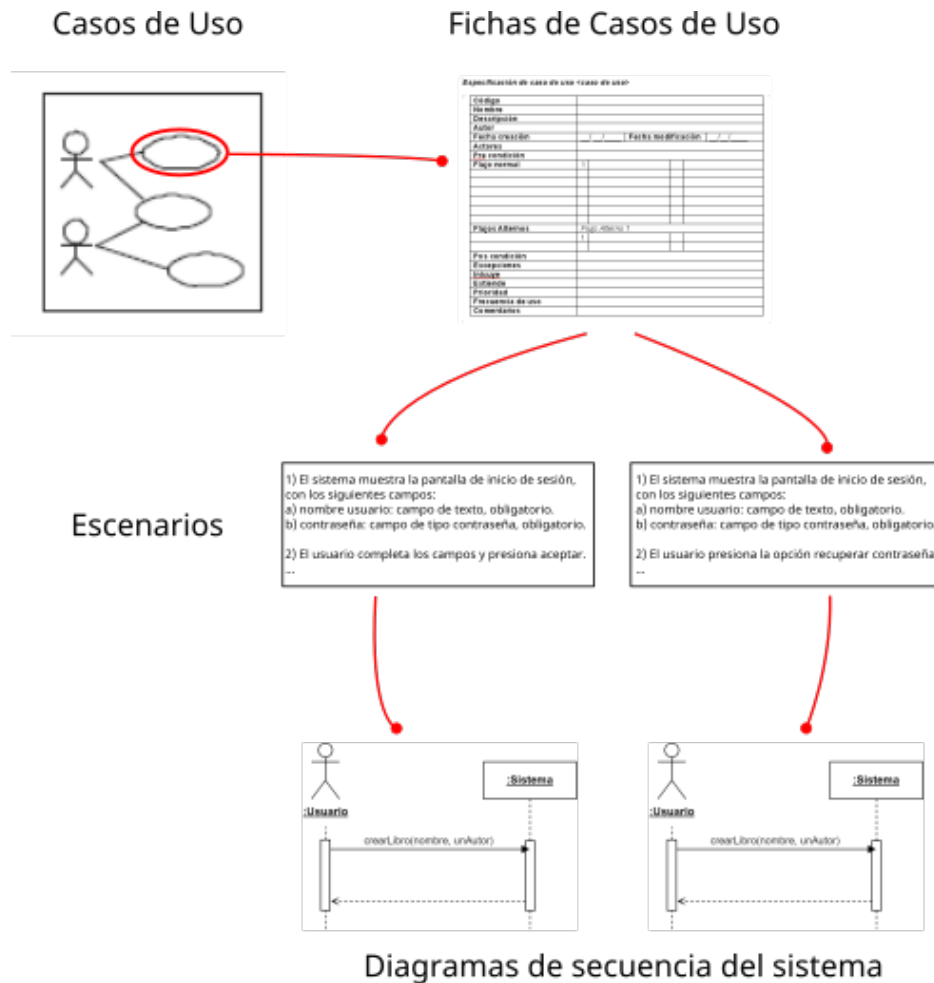


Figura 1. Relación entre los Cus y los DSS.

Entonces al modelar los DSS podemos ver como existe una relación entre los pasos de la ficha de especificación del CU que se reflejan en los pasos del escenario y los mensajes que vemos en el DSS ya que justamente este diagrama nos muestra más gráficamente la interacción Actor-Sistema. En la figura 2 podemos ver como los pasos de un escenario del CU tienen concordancia con los mensajes del DSS para este escenario.

Escenario simple del CU prestarLibro() para un Sistema de Gestión de Biblioteca

- 1 - El Usuario selecciona la opción "Nuevo préstamo"..
- 2 - El Sistema muestra el listado de socios.
- 3 - El Usuario selecciona el socio.
- 4 - El Sistema muestra el listado de Libros disponibles.
- 5 - El Usuario selecciona el libro a prestar.
- 6 - El sistema confirma el préstamo mostrando la fecha de devolución e imprime el ticket correspondiente.

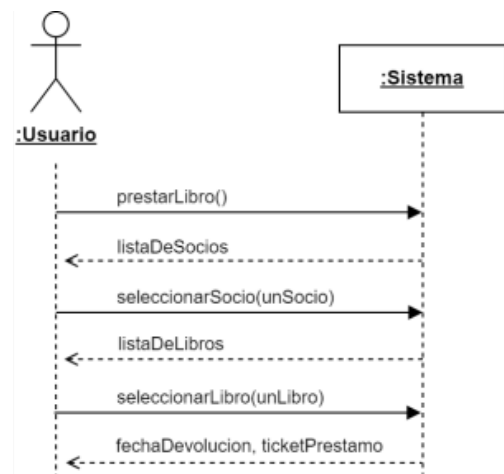


Figura 2. Pasos del escenario y DSS del mismo.

En los DSS (Figura 3) podemos ver como el actor y el sistema poseen líneas de vida que expresan el transcurso del tiempo hacia abajo, si bien no se utilizan unidades para medir el tiempo, si se puede afirmar que un mensaje que está debajo de otro ocurre luego. También se observan los mensajes modelados en ambos sentidos de acuerdo con quien es el emisor y quien el receptor.

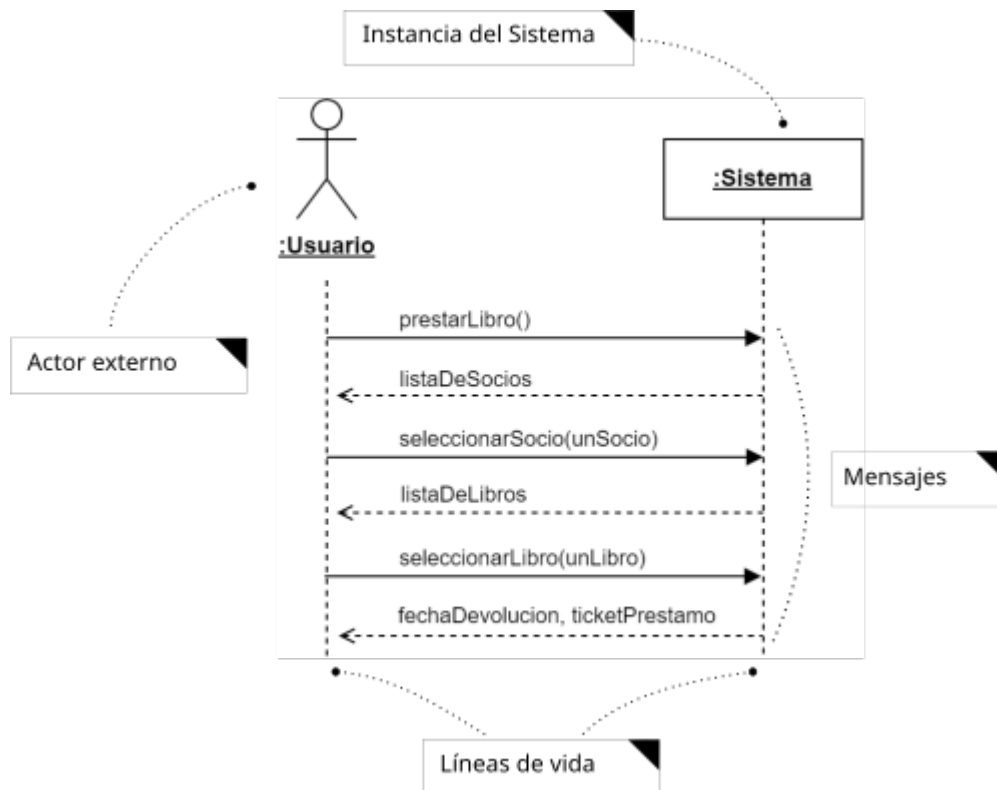


Figura 3. Ejemplo de DSS.

No es el objetivo de este apunte detallar demasiado el uso de los DSS, sino verlos como una introducción a los Diagramas de Secuencia.

Diagramas de secuencia

A diferencia de los DSS, los DS permiten representar explícitamente una secuencia de colaboraciones e interacciones entre las distintas instancias de clases que participan en un escenario de un determinado CU. Esto es como si expandiéramos la instancia **:Sistema** en un DSS y modeláramos lo que sucede internamente. Entonces de acuerdo con lo ya visto podemos concluir que para comenzar el modelado de los DSS es suficiente contar con el Diagrama de CUs y las fichas de especificación de CUs, mientras que para comenzar el modelado de los Diagramas de secuencia también necesitamos el Diagrama de Clases que nos provee de las clases, atributos y asociaciones que podemos utilizar (Figura 4).

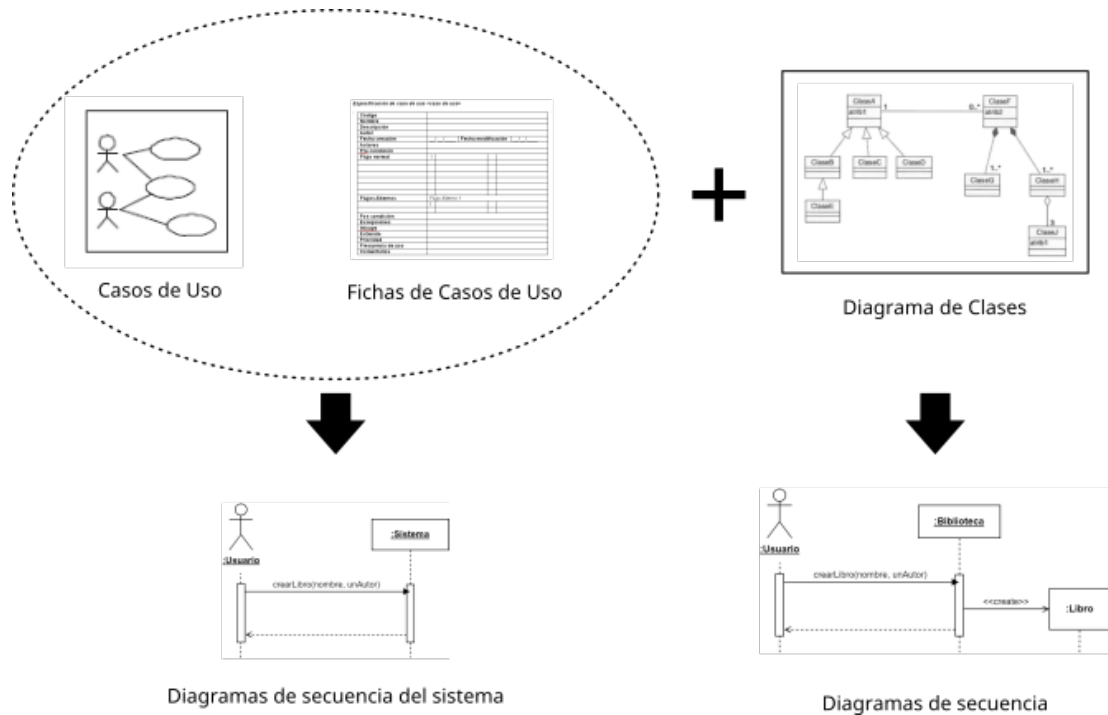


Figura 4. Diferencia entre los DSS y los DS.

Durante el desarrollo de esta etapa de modelado de comportamiento es probable que se descubran nuevos requerimientos, restricciones o clases. También al profundizar con los diagramas de secuencia se pueden empezar a descubrir más operaciones y atributos en las clases que participan. A partir de esto deberían actualizarse los diagramas de clases para mantener la consistencia entre los artefactos.

Para modelar los DS utilizaremos UML que nos brinda un conjunto de símbolos y notaciones para tal fin; se describen a continuación solo los elementos que se utilizan durante el curso.

Objetos, líneas de vida y cajas de activación

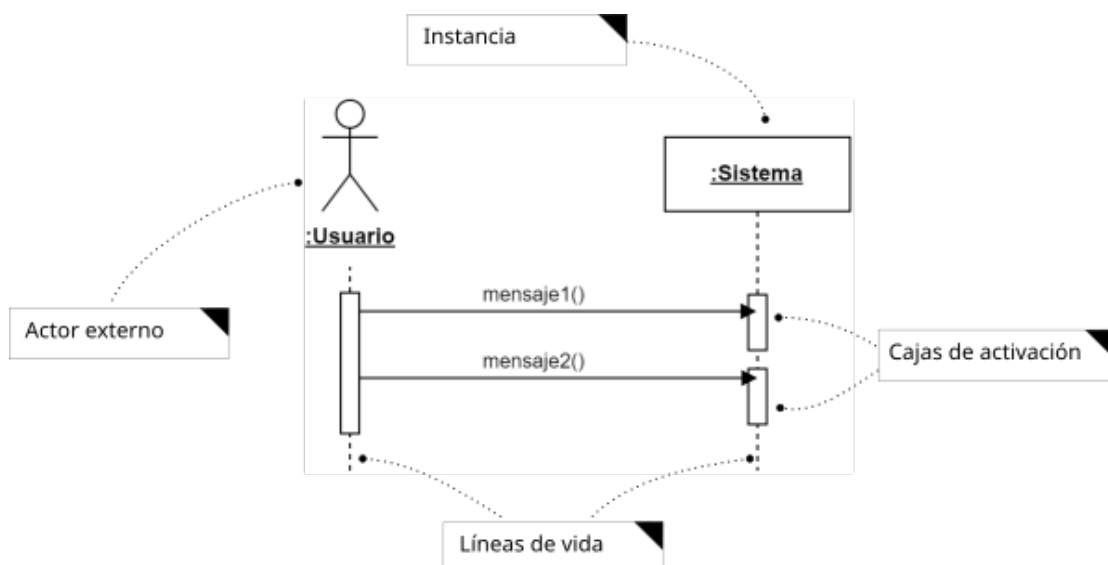


Figura 5. Objeto, línea de vida y caja de activación.

Como puede observarse en la Figura 5, cada objeto o actor representado tiene una línea de vida que se extiende mientras el objeto existe; además, mientras el objeto tiene el foco de control, sobre la línea de vida se grafica un rectángulo o caja de activación. Un foco de control o activación muestra el periodo de tiempo en el cual el objeto se encuentra ejecutando alguna operación.

Mensajes

Para expresar la interacción entre los actores externos y las instancias de clases o entre instancias de clases utilizaremos algunos de los tipos de mensajes que nos provee la especificación de UML (Tabla 1).

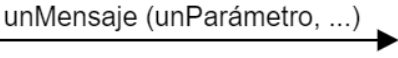
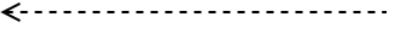
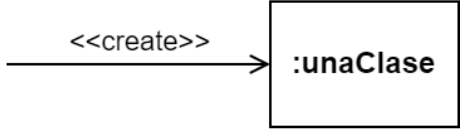
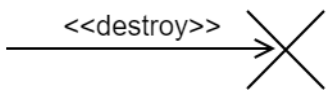
	Mensaje síncrono : El emisor espera hasta que regresa el control desde el receptor.
	Retorno de mensaje: El receptor de un mensaje anterior devuelve el control al emisor del mensaje.
	Creación de objeto: El emisor crea una instancia de la clase especificada por el receptor.
	Destrucción de objeto: El emisor destruye el receptor. Si la línea de vida tiene una línea vertical discontinua, se termina con una X.

Tabla 1. Tipos de mensajes.

Autodelegación

Se puede representar un mensaje que se envía de un objeto a él mismo utilizando una caja de activación anidada (Figura 6).

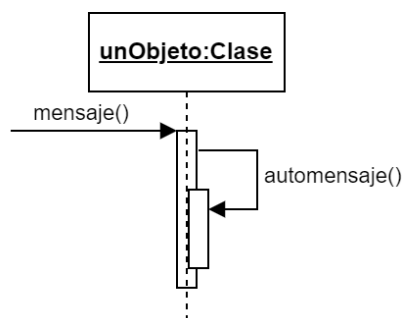


Figura 6. Ejemplo de auto delegación.

Creación de instancias

Se pueden crear nuevas instancias de objetos a partir del uso de mensajes `<<create>>`, como se muestra en la Figura 7.

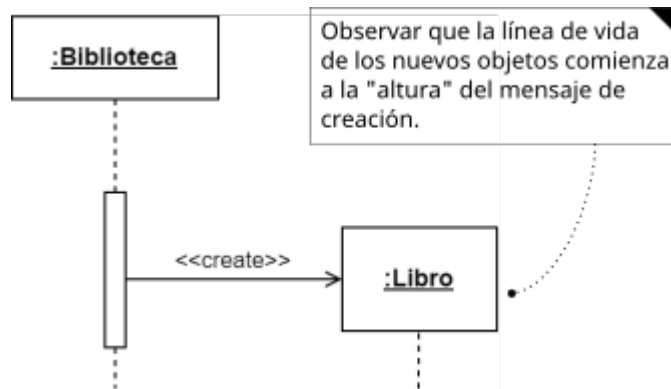


Figura 7. Creación de un nuevo objeto.

Destrucción de objetos

En algunas circunstancias es deseable mostrar la destrucción explícita de un objeto, la notación UML para las líneas de vida proporciona una forma para expresar esta destrucción utilizando un mensaje `<<destroy>>` como se ilustra en la Figura 8.

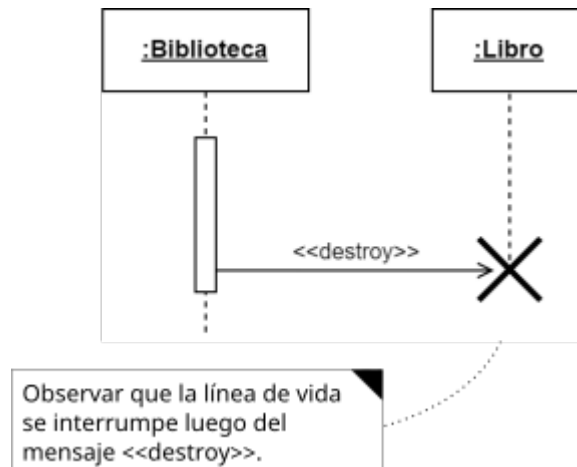
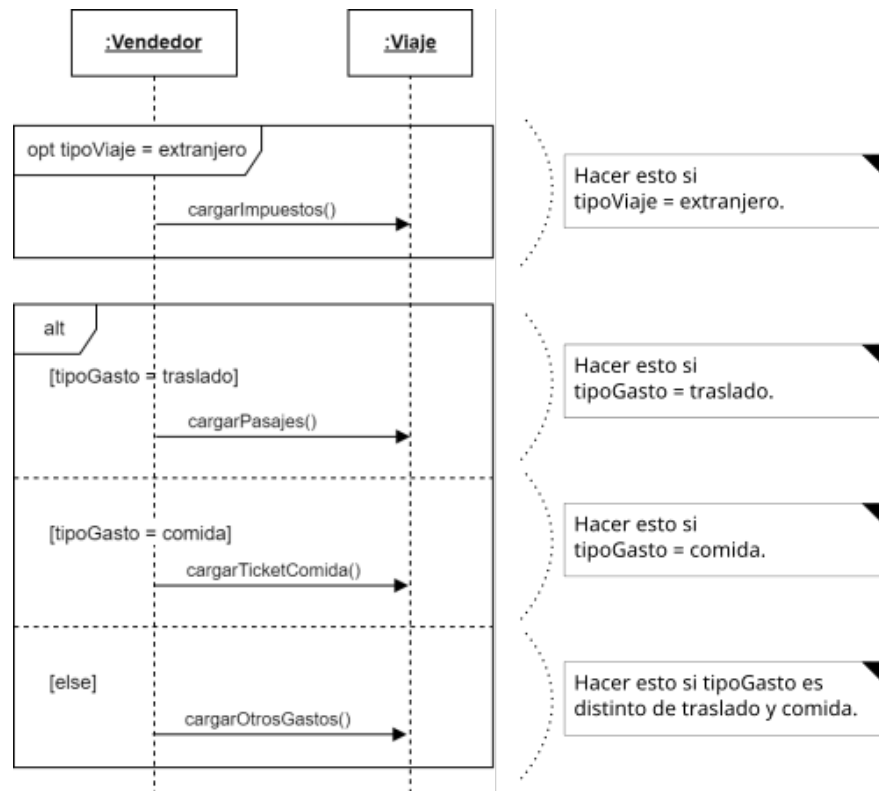


Figura 8. Destrucción de un nuevo objeto.

Fragmentos combinados y operadores

Los diagramas de secuencia pueden contener áreas demarcadas que encierran flujos, el comportamiento de estas áreas dependerá de su tipo y estas pueden cambiar la secuencia normal de mensajes haciendo que se repitan y ramifiquen dependiendo de un operador. Estos fragmentos combinados tienen un operador, uno o más operandos y cero o más condiciones de protección. En este curso se utilizarán cinco tipos de fragmentos combinados correspondientes con los operadores: *opt*, *alt*, *loop*, *break* y *ref*.

Ramificación con *opt* y *alt*Figura 9. Ejemplo de fragmentos *opt* y *alt*.

La presencia del operador *opt* indica que los mensajes contenidos en su único operando **se ejecutan si y solo si su condición de protección es verdadera** (Figura 9). De lo contrario la ejecución continúa luego del fragmento combinado, *opt* es el equivalente a utilizar en programación:

```

if (condición) then
    acción

```

El operador *alt* representa una opción entre alternativas. Cada uno de los operandos tiene su propia condición de protección y **se ejecutará si y solo si la condición de protección es verdadera**. Adicionalmente se puede agregar al final un operador *else* que se ejecuta si **ninguna de las condiciones de protección de los operadores anteriores es verdadera** (Figura 9). Es importante indicar que solo un operador de un fragmento *alt* puede ejecutarse por lo que las condiciones de protección de los operadores deben ser mutuamente exclusivas. El fragmento *alt* es el equivalente a utilizar en programación:

```

if (condición1) then
    operador 1
else if (condición2) then
    operador 2
else
    operando N

```

Iteración con *loop* y *break*

Los bucles e iteraciones son algo común en los algoritmos y para utilizarlos en los diagramas de secuencia se dispone de los fragmentos *loop* y *break*. El tipo de fragmento *loop* puede parecer complejo porque da solución a una gran variedad de tipos de bucle, se puede indicar que el *loop* itera desde un mínimo hasta un máximo y mientras una condición de protección es verdadera. El fragmento *break* interrumpe un *loop* agregando, si es necesario, la realización de mensajes dentro del fragmento. En la Figura 10 se muestra de forma simplificada la gestión de un Préstamo de Libros, el fragmento de *loop* sucede como máximo tres veces, mientras *codigoLibro* sea distinto de cero, al ocurrir una de estas condiciones se interrumpe la realización de mensajes dentro del fragmento y continúa el flujo debajo de la caja del mismo.

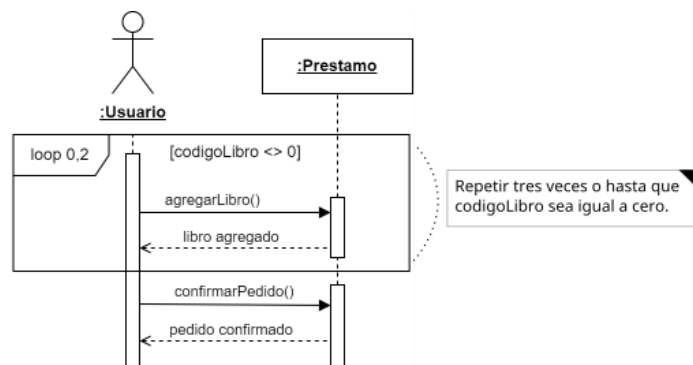


Figura 10. Ejemplo de fragmento loop.

La Figura 11 muestra un comportamiento similar, pero modelado de forma distinta para ejemplificar el uso del fragmento *loop* combinado con el fragmento *break*. El fragmento de *loop* también itera tres veces, pero si en el transcurso de alguna de las iteraciones el *codigoLibro* tiene valor 0 se cumple la condición del fragmento *break*, entonces el flujo ejecuta los mensajes de este fragmento para luego salir del fragmento *loop* y continuar con la realización de los mensajes que están debajo.

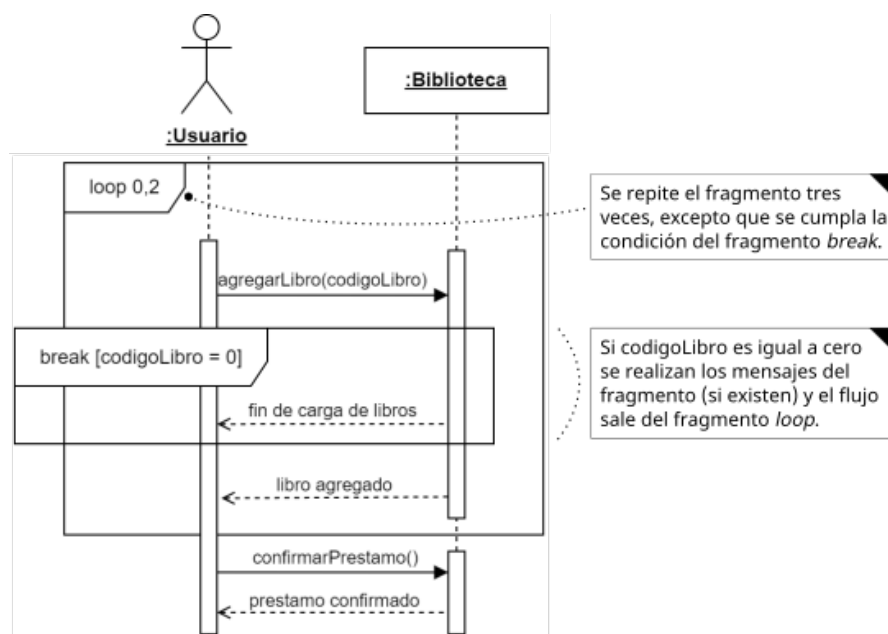


Figura 11. Ejemplo de fragmentos loop y break.

Fragmento de interacción con *ref*

A veces tenemos CUs que ocurren con frecuencia incluidos por otros CUs. Por ejemplo, podemos tener un CU *Iniciar Sesión* que ocurre al principio de todos los CUs que requieran que el usuario tenga la sesión iniciada. Para estas situaciones podemos hacer uso de un **fragmento de interacción que reutilizaremos cada vez que sea necesario**, de esta forma modelamos una sola vez el diagrama de secuencia del CU que se repite y luego con el fragmento *ref* lo referenciamos donde sea necesario.

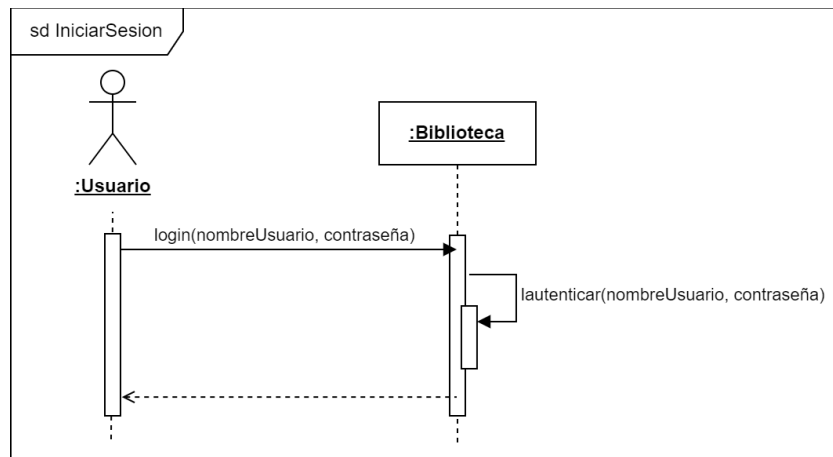


Figura 12. Ejemplo de fragmentos loop y break.

La Figura 12 muestra el diagrama de secuencia para el CU *Iniciar Sesión*, luego en la Figura 13 se observa cómo puede ser referenciado desde otro CU.

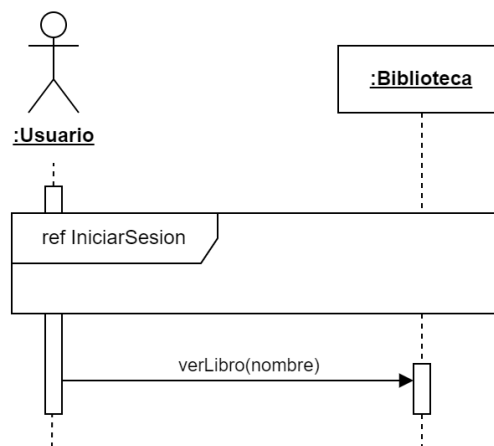


Figura 13. Reutilización de un diagrama de secuencia con *ref*.

Responsabilidades

Sin ahondar en las definiciones de responsabilidad y los distintos tipos de responsabilidades que pueden tener los objetos, en particular, al modelar interacciones entre objetos **debemos estar seguros de que el objeto receptor de un mensaje es responsable de ese tipo de acción**. En la mayoría de los casos puede ser bastante intuitivo reconocer al responsable de una interacción, pero a veces nos encontramos con problemas originados en el hecho de que, para esta instancia de Análisis, solo capturamos el comportamiento esencial de las clases que

hemos logrado identificar estudiando el dominio. Entonces nos pueden faltar interfaces y gestores que faciliten la tarea, pero estos corresponden a la etapa de diseño.

Es importante mantener la información que capturamos del dominio para determinar los objetos responsables de una acción determinada, dicho esto se citan a continuación algunos tipos de casos frecuentes donde se complica determinar un responsable.

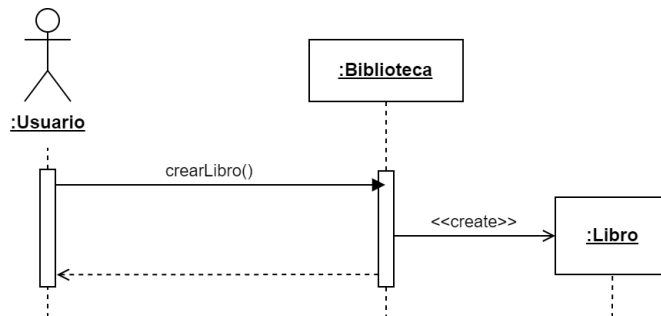


Figura 14. Las responsabilidades y las operaciones están relacionadas.

En la Figura 14 se puede ver que las instancias de Biblioteca tienen la responsabilidad de crear Libros, a esto se debe que pueden recibir un mensaje crearLibro() ya que poseen una operación definida para tal comportamiento. Si lo pensamos desde nuestro conocimiento del Dominio esto tiene su lógica, ya que podríamos definir a un Libro como parte de una Biblioteca (ya sea como una composición o agregación) y **tiene sentido que el “Todo” tenga la responsabilidad de crear las “Partes”**.

Patrón Experto en Información

Otra manera de encontrar el responsable de una operación es utilizar el patrón Experto en Información, este patrón propone asignar la responsabilidad a una instancia de la clase que tiene toda la información necesaria para realizar esta responsabilidad.

Podemos guiarnos mediante la pregunta *¿quién debería ser el responsable de conocer esta información?*, siguiendo con lo que recomienda el patrón se debe buscar entre las clases, la que tenga la información necesaria. Es posible que se encuentre una nueva clase que tenga toda la información necesaria y entonces se deba ampliar el diagrama de clases agregando las que correspondan de acuerdo con la información que se necesita para realizar la operación.

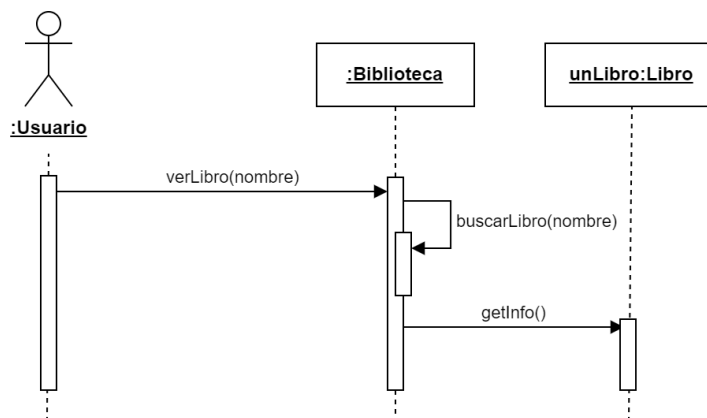


Figura 15. Ejemplo del patrón Experto de Información.

Analizando el diagrama de la Figura 15, la incógnita es *¿quién tiene la información necesaria para buscar un libro?*, entonces, *¿Qué información necesitamos para realizar esta búsqueda?* Para buscar un libro sin dudas necesitamos acceso al listado de libros existentes en la biblioteca, entonces la clase Biblioteca es un buen candidato para hacerse cargo de esta responsabilidad de acuerdo con el patrón Experto de Información.

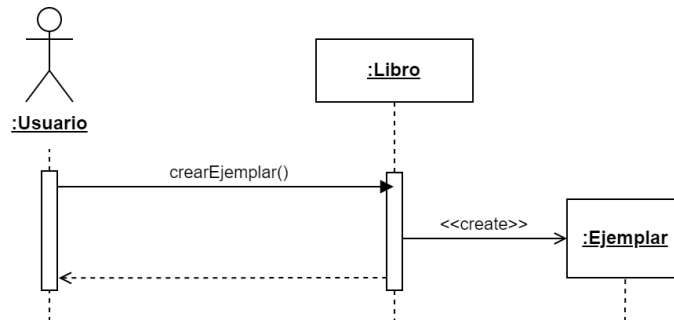


Figura 16. Ejemplo del patrón Experto de Información.

Siguiendo en la aplicación del patrón, podemos ver en la Figura 16 que a la pregunta *¿quién tiene la información necesaria para crear un ejemplar?*, la clase Libro es la mejor opción ya que tiene la información necesaria. Esto tiene su lógica ya que, si pensamos a la clase ejemplar como una materialización de la clase libro, es correcto que un concepto más abstracto pueda crear instancias de un objeto más concreto.

Por otro lado, para este ejemplo se puede ampliar la discusión preguntando *¿la clase Biblioteca no tiene la información para crear ejemplares?* Por cierto, la respuesta es afirmativa, **podemos escoger como el experto a la clase Biblioteca, pero esto incrementaría innecesariamente el acoplamiento de la clase**. Entonces, podemos decir que, ante la postulación de varios Expertos es bueno **escoger el que menos afecte al acoplamiento y la cohesión**.

Algunas veces surgen con la aplicación de este patrón clases que representan una gran porción o todo el dominio de información, esto lleva a Expertos de Información con problemas de acoplamiento y cohesión, no se ampliará en este apunte sobre estos problemas y como solucionarlos, prefiriendo su utilización que por lo general se acerca mucho a nuestra concepción sobre como se organiza el dominio en el mundo real.

Bibliografía

Arlow, J., Neustadt, I. 2006. *UML 2*. Anaya multimedia.

Larman, C. y Valle, B.M. 2003. *UML y patrones*. Pearson Educación.

OMG – UML. 2017. *Unified Modeling Language® (OMG UML®) Version 2.5.1*, disponible en <https://www.omg.org/spec/UML/>